

PowerSkills 实践指南

把资深电力工程师的经验装进 AI 智能体

仓库: [Power-Agent/PowerSkills](#) | **许可:** MIT | **Stars:** 66 **定位:** Agent Skills 技能层, 21 个技能 (11 个软件工作流 + 10 个工程处置剧本) **依赖:** [PowerMCP](#) (提供底层工具) **适用读者:** 电力系统工程师、电网运维与规划人员、电力 AI 应用开发者

一、核心问题：为什么有了工具还不够

1.1 一个真实会发生的错误

假设你给智能体装好了 PowerMCP，它拥有 pandapower、PyPSA、surge 等全套工具。然后你说：

"看看这个案例安不安全。"

一个**没有经验**的智能体会怎么做？它很可能直接调用 `run_contingency_analysis()`，然后给你一堆 N-1 结果。

这错在哪？

如果基态潮流本身就没收敛，或者基态就已经有线路过载 105%，那么 N-1 的结果**毫无意义**——你连正常工况都没搞清楚，谈什么故障工况？

这正是**初级工程师的典型错误**。而资深工程师的第一反应永远是：

"先确认基态正常，再看是否值得做故障分析。"

1.2 PowerSkills 的定位

层	项目	解决的问题
工具层	PowerMCP	智能体 能不能 调用软件
技能层	PowerSkills	智能体 知不知道 怎么做

一句话理解：PowerMCP 给了智能体"手"，PowerSkills 给它"经验"。

1.3 两大插件

插件	内容	技能数	回答的问题
<code>powerskills-tool</code>	软件渐进式使用工作流	11	"这个软件该怎么用？"
<code>powerskills-engineering</code>	资深工程师处置剧本	10	"遇到这个问题该怎么办？"

二、理解技能 (Skill) 是什么

在深入之前，先搞清楚"技能"这种技术形态。

2.1 技能 = 一个带 SKILL.md 的文件夹

```
voltage-violation-mitigation/
├─ SKILL.md           ← 核心：技能说明书
├─ scripts/           ← 可选：配套脚本
├─ references/         ← 可选：参考资料
└─ requirements.txt    ← 可选：依赖
```

SKILL.md 包含 YAML 头部（元数据）+ Markdown 正文（工作流）：

```
---
name: voltage-violation-mitigation
description: Senior power-engineer playbook for voltage violations.
    Use whenever a base-case or contingency study shows low voltage, high voltage,
    weak reactive support, or tap exhaustion – including escalations from a tool
    skill where bus voltage falls below 0.95 pu or rises above 1.05 pu...
---

# Voltage violation mitigation

Start only after the violated buses, limits, and study condition are known.

## Preferred action order
1. Confirm the violation is real...
2. Use existing reactive support first...
```

2.2 关键机制：description 决定"何时被激活"

技能的 **description** 字段**不是给人看的简介，而是给模型看的触发条件**。模型会读取所有技能的 description，判断当前任务该加载哪个。

这也是为什么 PowerSkills 的 description 写得非常长——它要穷举各种可能的触发场景：

```
"Use whenever a base-case or contingency study shows low voltage, high voltage, weak reactive support, or tap
exhaustion — including escalations from a tool skill where bus voltage falls below 0.95 pu or rises above
1.05 pu in PowerWorld, PSS/E, PSLF, OpenDSS, pandapower, PyPSA, or surge. Triggers on "voltage too low",
"voltage too high", "add reactive support", or "buses out of limits"."
```

即使用户只说"电压有点低"，甚至只是把一份显示 0.94 pu 的结果丢过来，技能也能被正确激活。

三、11 个软件 workflow 技能

3.1 统一的"工具阶梯"设计

每个软件技能都强制遵循同一个渐进式结构：

- ① 加载/打开案例
- ② 检查模型（总线数、元件、拓扑）
- ③ 求解基态（潮流） ← 必须通过这一关
- ④ 修改参数（受控的变更）
- ⑤ 高级研究（故障、灵敏度、优化、动态）

核心纪律：在基态可信之前，不要碰高级研究。

以 pandapower 技能为例，原文明确写道：

"Do not run contingencies before checking the base-case voltages and loading."（在检查基态电压和负载率之前，不要运行故障分析。）

3.2 完整技能清单

技能	软件领域	首选工具	后期工具
pandapower	稳态 AC 分析	<code>load_network</code> 、 <code>get_network_info</code> 、 <code>run_power_flow</code>	<code>run_contingency_analysis</code>
PyPSA	规划、OPF、 扩张	<code>load_network</code> / <code>create_network</code> 、 网络检查	<code>optimize_network</code> 、 <code>optimize_investment</code>
surge	输电分析全流程	<code>load_built_in_case</code> 、 <code>get_network_info</code> 、 <code>run_ac_power_flow</code>	<code>compute_ptdf</code> 、 <code>run_dc_opf</code> 、 <code>run_scuc</code> 、 <code>compute_nerc_atc</code>
PowerWorld	稳态分析与灵敏度	<code>open_case</code> 、 <code>run_powerflow</code> 、结果查询	故障分析、参数修改、PTDF/LODF/Jacobian
PSS/E	基态与 API 引导研究	<code>open_case</code> 、 <code>solve_case</code>	<code>lookup_psspy_command</code> 、 <code>run_psspy_command</code>
PSLF	输电潮流与故障	<code>open_case</code> 、 <code>solve_case</code> 、违规检查	拓扑/设备编辑、 <code>run_contingency_analysis</code>
OpenDSS	配电网馈线	<code>compile_and_solve</code> 、 <code>get_total_power</code> 、 <code>get_bus_voltages</code>	<code>set_load_multiplier</code> 、 <code>run_daily_energy_meter</code> 、 <code>get_harmonic_results</code>
ANDES	动态安全与小信号	<code>run_power_flow</code> 、 <code>get_system_info</code>	<code>run_eigenvalue_analysis</code> 、 <code>run_time_domain_simulation</code>
Egret	市场与运行优化	<code>solve_dc_opf</code>	<code>solve_ac_opf</code> 、 <code>solve_unit_commitment_problem</code>
LTSpice	电路仿真	<code>create_simulation_session</code> 、 <code>run_simulation</code>	<code>list_available_traces</code> 、绘图、GUI 查看
potpourri	配电网 AC/DC 与多时段 OPF (含柔性资源与储能)	<code>scripts/inspect_case.py</code> 、 <code>solve_opf.py --list-solvers</code>	<code>--formulation dc</code> → <code>--formulation ac</code> 、 <code>--horizon N</code> <code>--battery-penetration</code>

关于 **potpourri**: 这是唯一**没有 PowerMCP 服务器**的技能，通过它自带的 Python 脚本驱动它的 Python API。它基于 Pyomo 在 pandapower 网络上构建 AC/DC OPF 模型。

3.3 技能附带的资产

部分技能自带可运行的脚本、参考资料和测试案例，这是实践中最有价值的部分：

技能	附带资产
pandapower	<code>case39.json</code> （测试网络）、 <code>network_analysis.py</code> 、 <code>contingency_analysis.py</code> 、 <code>API_REFERENCE.md</code> 、 <code>EXAMPLES.md</code>
PyPSA	<code>case39.nc</code> （测试网络）、 <code>network_analysis.py</code> 、 <code>optimization_analysis.py</code> 、 <code>expansion_analysis.py</code> 、 <code>contingency_analysis.py</code> 、 <code>build_case39.py</code>
surge	8 个脚本（ <code>network_analysis</code> 、 <code>sensitivity_analysis</code> 、 <code>contingency_analysis</code> 、 <code>contingency_ranking</code> 、 <code>opf_analysis</code> 、 <code>dispatch_analysis</code> 、 <code>transfer_capability</code> 、 <code>test_scripts</code> ）、 <code>API_REFERENCE.md</code> （44 工具目录）、 <code>SOLVER_GUIDE.md</code> 、 <code>DISPATCH_REQUEST_SCHEMA.json</code> 、 <code>evals/</code> （评测集）
potpourri	<code>inspect_case.py</code> 、 <code>solve_opf.py</code> 、 <code>API_REFERENCE.md</code> 、 <code>FORMULATION_GUIDE.md</code> 、 <code>SOLVER_GUIDE.md</code> 、 <code>VALIDATION_GUIDE.md</code>

实践提示：surge 技能是唯一自带 `evals/` 评测目录的（含 `evals.json`、`mcp_config.example.json`、`run_eval_with_mcp.py`）。如果你要评估自己的模型在电力任务上的表现，这是一个现成的起点。

四、10 个工程处置剧本（重点）

这是 PowerSkills 最有价值的部分——把资深工程师的隐性经验显式编码成结构化流程。

4.1 全景

处置技能	触发场景	核心动作序列
<code>voltage-violation-mitigation</code>	电压偏低/偏高、无功支撑薄弱、分接头用尽	确认 → 用现有无功 → 减少无功输送 → 减负荷
<code>thermal-overload-mitigation</code>	线路或变压器过载	确认 → 再调度 → 拓扑重构 → 移相器 → 加固
<code>contingency-mitigation</code>	N-1/N-2 违规、故障后孤岛	核对定义 → 按走廊分组 → 预防性动作 → 纠正性动作
<code>dynamic-stability-mitigation</code>	阻尼差、暂态失稳、电压恢复慢	确认模型 → 减载 → 改进控制 → 改进保护
<code>frequency-response-mitigation</code>	低惯量、RoCoF 差、一次响应弱	确认设置 → 增加旋转备用 → 提高惯量 → 修控制参数
<code>convergence-failure-mitigation</code>	潮流发散	排除数据问题 → 检查孤岛/平衡机 → 简化求解 → 逐步恢复
<code>short-circuit-mitigation</code>	短路电流超断路器额定	确认数据 → 操作手段 → 增加阻抗 → 故障电流限制器 → 换断路器
<code>interconnection-impact-mitigation</code>	新增发电/储能/大负荷并网筛查	确定场景集 → 稳态筛查 → 弱网 SCR 检查 → 无功能力 → 组合措施
<code>der-hosting-capacity-mitigation</code>	DER 引起电压抬升、反向潮流、保护失敏	确认约束类型 → 逆变器能力 → 馈线控制 → 保护复核 → 加固
<code>operations-planning-mitigation</code>	OPF/UC 不可行、弃电高、阻塞	修数据 → 简化筛选 → 增加灵活性 → 缓解阻塞 → 谨慎放松约束

4.2 深入解读：电压越限处置

触发：基态或故障研究中出现低电压、高电压、无功支撑薄弱或分接头用尽。

动作顺序（严格按此顺序）：

1. 确认违规是真的
 - 已求解的案例？限值正确？发电机 Q 限值合理？
 - AVR / Volt-VAR 控制正常工作？远方调节目标正确？
2. 先用现有无功资源
 - 发电机电压设定值、空闲 LTC、投切并联装置或电容器
 - 启用 SVC / STATCOM / 逆变器 Volt-VAR 响应
3. 减少无功输送
 - 再调度有功更靠近负荷；减少受压力断面的输入；重构拓扑
4. 仍不解决才降输电或负荷，并识别加固需求

高电压场景单独处理：

- 谨慎降低电压目标值
- 切除电容器，或投入电抗器
- 让发电机或逆变器吸收无功（在能力范围内）
- 轻载和故障工况要单独复核

分软件的操作提示：

软件	优先操作
PowerWorld / PSLF / PSSE	调整并联装置、分接头、发电机控制，然后重新求解
OpenDSS	调压器、电容器组、逆变器控制 先于 馈线重建
pandapower	并联装置、分接头设置、发电机无功支撑是第一批杠杆
PyPSA / Egret	用来 筛查 调度或输电方案，再用 AC 工具 验证 电压

最后一条特别值得注意：PyPSA 和 Egret 是做规划调度的，**它们不是详细无功处置的最终权威**。用它们筛选方案，但电压一定要回到 AC 工具验证。这是一个很容易被忽略的边界。

交付物要求：违规母线与工况、尝试或推荐的处置序列、**第一个在不制造更严重问题的前提下解决问题的动作**。

4.3 深入解读：热过载处置（含完整算例）

动作顺序：

1. 确认过载是真的（正确的额定值集、正确的停运状态、已求解的基态或故障态）
2. 再调度发电或输电模式，卸载该走廊
3. 在运行允许时重构拓扑或切换
4. 仅在灵敏度证据支持时使用移相器、HVDC 计划或变压器控制
5. 若问题持续，转向换线、增容或新建
6. 每个动作后重新求解确认过载已缓解；若没有，继续下一步

完整算例（这是理解"用灵敏度指导动作"的最佳示例）：

线路 A-B 在 Line C-D 发生 N-1 停运后达到 **105%** 额定值。

从 G3 到 G5 的转移在线路 A-B 上的 **PTDF 为 -0.18 MW/MW**。

因此把 **40 MW** 从 G3 转移到 G5，可卸载 A-B 约 $40 \times 0.18 \approx 7 \text{ MW}$ ，使其降至 **94%**。

重新求解**全部受监视故障集**，确认该再调度未在其他地方制造新违规。

这个算例的价值在于它展示了完整的推理链：识别过载 → 查灵敏度 → 计算所需转移量 → 执行 → **全故障集复核**。

最后一步最关键。只验证最严重的那个故障是不够的——再调度可能把问题转移到别的故障上去。

工作规则： - 区分**正常额定值**问题与**紧急额定值**问题 - 只修**最小元件集合**来缓解受监视的过载簇 - 候选方案要在**所有关键受监视故障**上验证，不只是最坏情况 - 软件提供 PTDF / LODF 类灵敏度时就用它们

4.4 深入解读：故障处置（按"族"思考）

动作顺序：

1. 核对故障定义、受监视元件与限值
2. 把关键故障按共同走廊、断面或薄弱区域分组 ← 关键步骤
3. 先做预防性动作：再调度、并联装置、调压、拓扑变更
4. 再评估纠正性动作：故障后切换、再调度、带明确触发逻辑的 RAS
5. 任何候选方案后重跑全部受监视集；仍有违规就回到第 3 步
6. 甩负荷只作为最后手段，并且要明确说明
7. 反复出现的问题，推荐加固或修订运行导则

完整算例：

Line X-Y 的 N-1 停运导致： - Line A-B 过载至 **108%** 额定值 - 母线 C 电压跌至 **0.91 pu**

分组分析发现两个违规都在**同一条东西向走廊**上，所以**一个预防性动作可以覆盖整族故障**： - 从 G1 向 G4 转移 **60 MW** → 消除热违规 - 投入母线 C 的 **50 Mvar 电容器组** → 电压恢复至 **0.97 pu**

重跑每个受监视故障，确认无新违规。

设计精髓：先分组、再动手。如果不做分组，工程师可能会针对每个故障单独设计措施，结果做了三个动作解决的问题，实际上一个动作就够了。

原文的表述是："Prefer actions that improve a family of contingencies rather than a single outage."（宁可选择能改善一族故障的动作，而不是只改善单个停运。）

4.5 深入解读：收敛失败处置

这个技能的独特之处在于——**目标是搞清"为什么"失败，而不只是强行让它收敛。**

动作顺序：

1. 先排除数据问题
 - 孤岛或失电母线、零或荒谬的支路阻抗
 - 平衡机缺失或位置错误
 - 变压器分接头与相角数据
 - 负荷/发电总量无法平衡
2. 检查孤岛与平衡覆盖
 - 每个带电孤岛都需要一个 MW 可行的平衡机
 - 先用工具的孤岛/拓扑报告，再去动求解器设置
3. 简化求解，再逐步恢复
 - 平启动 vs 热启动；锁定分接头/投切并联/移相器；放宽发电机 Q 限值
 - 有了解之后，再逐步重新启用控制
4. 从更简单的点播种
 - DC 解、之前求解过的相邻案例、降应力版本
 - 然后逐步把负荷或输电推向目标
5. 若仅在接近目标应力水平时才失败，视其为可行性边界而非数值 bug
 - 降低输电或负荷、增加无功支撑、标记可能的电压崩溃临近

两条最有价值的工作规则：

"The largest mismatch bus tells you where to look — start there, not with global solver settings."（最大失配母线告诉你去哪里找——从那里开始，而不是从全局求解器设置开始。）

"Distinguish 'the math failed' from 'the operating point does not exist' — the corrective actions are completely different."（区分"数学失败了"和"运行点根本不存在"——两者的纠正措施完全不同。）

第二句话是整套技能里最重要的工程判断之一。如果运行点物理上就不存在（接近电压崩溃点），你无论怎么调求解器参数都解不出来，正确做法是降低输电或增加无功支撑。而如果只是数值脆弱性，调整启动方式或锁定控制就能解决。混淆这两者会导致大量无效调试。

交付物要求：失败分类（数据错误 / 控制交互 / 数值脆弱性 / 物理不可行）、让案例求解出来的具体改动、以及**遗留的真实性警告**（锁定控制、放宽限值）和需要复检的内容。

最后一条很关键："A case solved with controls locked or Q-limits relaxed is not finished; restore realism before reporting results from it."（用锁定控制或放宽 Q 限值求解出来的案例不算完成；报告结果前必须恢复真实性。）

4.6 深入解读：频率响应处置

前提：必须先明确设计事件和失败指标——是**哪个故障**，以及 **nadir（最低点） / RoCoF（频率变化率） / 稳态频率**三者中**哪一个**违规。

动作顺序（顺序很讲究）：

1. 确认研究设置可信
 - 最大可信损失、负荷阻尼假设
 - 调速器模型确实启用且响应（死区、基荷标志）
 - 所研究调度的惯量
2. 先增加可响应备用
 - 开停机以创造调速器备用
 - 启用储能、IBR、需求响应的快速频率响应
 - ← 前几秒内响应的 MW 最重要
3. 备用不足时提高惯量
 - 增加同步机组、带飞轮的同步调相机、低惯量时段的必开规则
4. 修控制参数
 - 下垂、死区、响应撤回行为——让一次响应是持续的而非瞬态的
5. 最后才是重新协调低频减载
 - UFLS 是兜底，不是解决方案本身

四条工作规则：

规则	含义
评估 最差惯量小时	高新能源、轻负荷，而非平均调度
系统级指标筛查 → 时域确认	用 ANDES / PSS/E 确认关键工况后再声称有效
跟踪响应 轨迹	nadir 好但响应快速撤回，仍可能不满足稳态频率要求
频率遏制与振荡阻尼是不同问题	阻尼差的响应转给 dynamic-stability-mitigation

第 3 条容易被忽略但很重要：一个"最低点看起来不错"的响应，如果恢复得太快，稳态频率可能仍然超标。要看整条曲线，不能只看一个点。

4.7 深入解读：DER 承载能力（最"反直觉"的一个）

开篇第一句就点出了核心：

"Start by identifying the binding constraint, **not by assuming it.**"（先识别约束是什么，**不要假设。**）

动作顺序：

1. 用最小负荷/最大发电快照确认约束类型 (OpenDSS、pandapower)
 - 沿馈线的电压抬升、变电站反向潮流、元件负载率、调压器行为
2. 先用逆变器能力
 - Volt-VAR 和 Volt-Watt 曲线、固定功率因数设置、出力限制
 - ← 便宜、快速，通常能恢复可观的裕度
3. 再调整馈线控制
 - 调压器和 LTC 设定值 (含反向功率模式行为)、电容器投切与新潮流的协调
4. 复核保护
 - DER 馈入导致的继电器范围缩减、同情跳闸、熔断器-重合器配合、高 DER 出力下的防孤岛行为
5. 最后才加固
 - 换线、加装调压器、相平衡、负荷/DER 转移
 - ← 要量化每个升级带来的承载能力增益，让最便宜的充分方案胜出

四条工作规则（每一条都是实战经验）：

规则	说明
研究最小负荷、最大发电快照	峰值负荷研究 系统地漏掉 DER 约束
弱馈线上电压抬升和保护通常 先于 热约束	不要只检查负载率就收工
验证修复要跨 日曲线	修好正午的 Volt-VAR 在黄昏可能失灵
传统调压器在反向潮流下可能误动作	依赖它们前先显式检查模式

第一条最反直觉：很多人做 DER 研究时会习惯性地看峰值负荷工况，但那恰恰是**最不容易出问题**的工况。DER 的问题是**发电多、负荷少**——所以要看最小负荷、最大发电的快照。

4.8 深入解读：并网影响评估

动作顺序：

1. 确定研究场景集
 - 峰荷与低谷案例、高新能源调度
 - 必须建模为在运的****先前排队项目****
2. 稳态筛查
 - 基态与 N-1 潮流、POI 附近的元件负载与电压筛查
 - 违规转给 thermal-overload / voltage-violation 处置技能
3. 检查 IBR 的弱网风险
 - POI 处的短路比 (SCR)
 - SCR 低于约 3 提示控制交互与稳定风险；低于 2 为临界
 - 考虑构网型控制、同步调相机或 POI 重新选址
4. 核验无功能力与电压控制
 - POI 处的功率因数范围与电压穿越能力
5. 确定约束并组装措施集
 - 网络升级、出力限制或弃电协议、无功设备、分阶段并网
 - ← 以最小的能清除约束筛查的集合为准

关键工作规则：

"Separate "needed to connect safely" from "needed to deliver full output" — reliability upgrades and deliverability upgrades have different urgency and cost allocation."（把"安全并网所需"与"满发所需"分开——可靠性升级和输送能力升级的紧迫性和成本分摊是不同的。）

这是一条**商务与技术交界处**的重要判断。混淆两者会导致不必要的过度投资——为了安全并网必须做的升级，和为了满发才需要的升级，在工期与费用分摊上完全不是一回事。

4.9 其他技能的要点提炼

短路电流处置： - 开篇提醒："over-duty findings are often data findings"（过负荷发现往往其实是数据发现）——先核对应次暂态数据、电动机贡献、故障前电压假设、故障类型、X/R - 必须同时检查**最大**（全电源投入）和**最小**（可信停运）故障工况：保护整定需要最小值，断路器能力需要最大值 - **单相接地故障的电流可能超过三相故障**（靠近接地变压器组时）——不能只筛查三相故障 - 母线分裂会改变潮流转移路径，拓扑修改后**必须重跑故障筛查**

运筹规划处置： - "Do not call a system infeasible before checking data consistency"（在检查数据一致性之前，不要说系统不可行） - 求解顺序：**DC 先于 AC，调度先于机组组合，运行先于扩张** - 只有在明确惩罚和清晰说明时才放松约束或加松弛变量 - 区分**经济性阻塞**与**物理 AC 不可行**

动态稳定处置： - "Use eigenvalues to screen and time-domain simulation to confirm"（用特征值筛查，用时域仿真确认） - 必须区分**功角失稳、振荡失稳、电压恢复问题**——三者的修复顺序完全不同 - 重大控制或拓扑变更后要**重新检查稳态调度**

五、核心机制：数值驱动的升级触发器

5.1 这是整套技能最精妙的设计

每个**软件技能**结尾都必须附带一张**升级触发器（Escalation Triggers）**表格，把具体数值观测映射到对应的工程处置技能。

设计要点：**由数字驱动，而不是靠感觉。**

原文："so the handoff is driven by **numbers, not vibes.**"

5.2 完整触发器对照表

观测到的数值	升级至
任何母线 <code>vm_pu</code> < 0.95 或 > 1.05	<code>voltage-violation-mitigation</code>
任何支路负载率 > 100% 额定值	<code>thermal-overload-mitigation</code>
<code>run_n1_*</code> 返回 ≥ 1 个约束性故障	<code>contingency-mitigation</code>
<code>get_islands()</code> 在故障后返回 > 1 个孤岛	<code>contingency-mitigation</code>
SCED/SCUC 不可行、备用短缺或 LMP 波动	<code>operations-planning-mitigation</code>
<code>run_ac_power_flow</code> 基态不收敛	<code>convergence-failure-mitigation</code>
研究涉及 POI 处新增发电机、储能或大负荷接入	<code>interconnection-impact-mitigation</code>

多个触发器同时命中时的优先级：

电压 → 热 → 故障

为什么要规定优先级：如果同时有电压违规和过载，先处理电压——因为许多缓解电压的手段（如投切并联装置、调整分接头）**也会改变潮流分布**，可能顺带缓解过载。反过来则不一定成立。

5.3 各软件的触发器差异

不同软件的技能会根据其能力特点调整触发器：

软件	特有触发器
surge	孤岛检测（ <code>get_islands()</code> > 1）、SCED/SCUC 不可行、LMP 波动
ANDES	特征值阻尼比 < 约 5% 或任何右半平面模式；时域出现角度增长、持续振荡、电压恢复失败
PSSE	故障电流接近或超过断路器开断额定值
OpenDSS	编译成功但求解发散
potpourri	<code>termination_condition</code> 为 <code>infeasible</code> / <code>unbounded</code> ，或弃电超过可用新能源的 5%
Egret	OPF/UC 不可行、备用短缺、成本异常偏高

5.4 最重要的纪律：必须报出具体数字

所有技能都反复强调同一条规则：

"Report the specific numbers that triggered the escalation (bus + vm_pu, branch + loading %, contingency label).
Do not say "violations exist" without values."（报告触发升级的具体数字——母线+电压值、支路+负载率、故障标签。**不要在没有数值的情况下说"存在违规"。**）

这条规则的深层意义：**它让"AI 的结论"变得可核查。**

当模型说"存在违规"，你无法判断它是真算过还是编的。当它说"母线 7 电压 0.9473 pu，低于 0.95 pu 下限"，你可以立刻去案例里核对。这是把 AI 输出变成**工程可验证陈述**的关键一步。

5.5 完整的交接链路

把所有机制串起来看：

用户提问

↓

【软件技能激活】按工具阶梯执行

① 加载案例 → ② 检查模型 → ③ 求解基态

↓

求解输出数值（母线电压、负载率、故障标签）

↓

【数值驱动的升级判断】

0.9473 pu < 0.95 → 命中电压触发器

105.5% > 100% → 命中热过载触发器

↓

【处置技能激活】按优先级执行

电压处置优先 → 投切并联/调压

热过载处置次之 → 再调度/拓扑

↓

输出：违规母线与工况 + 处置序列 + 首个有效动作 + 残留风险

六、安装与部署

6.1 Claude Code（推荐）

```
# 添加插件市场
/plugin marketplace add Power-Agent/PowerSkills

# 安装两个插件
/plugin install powerskills-tool@powerskills
/plugin install powerskills-engineering@powerskills
```

从本地克隆安装（而非 GitHub）：

```
/plugin marketplace add ./
```

后续用 `/plugin` 统一管理。

6.2 Codex CLI

Codex 从 `.agents/skills/`（从工作目录向上扫描到仓库根）和 `~/.codex/skills/`（全局）发现技能。**直接复制即可，无需转换：**

```
# 项目级：只给当前仓库用
mkdir -p .agents/skills
cp -R /path/to/PowerSkills/powerskills-tool/skills/* .agents/skills/

# 全局：所有项目都能用
mkdir -p ~/.codex/skills
cp -R /path/to/PowerSkills/powerskills-tool/skills/* ~/.codex/skills/
cp -R /path/to/PowerSkills/powerskills-engineering/skills/* ~/.codex/skills/
```

6.3 Claude Desktop / claude.ai

技能通过 ZIP 上传，一次一个：

1. 开启 **Settings → Capabilities → Code execution**（技能在代码执行沙箱中运行）
2. 打开 **Settings → Capabilities → Skills → Upload skill**
3. 上传单个技能文件夹的 ZIP，**技能文件夹必须在 ZIP 根目录**

```
cd powerskills-tool/skills
zip -r pandapower.zip pandapower
```

上传后启用开关。对你需要的每个技能重复此流程。

平台差异提醒： Claude Desktop 的技能上传比较繁琐（逐个上传）。如果你需要整套技能，**Claude Code 或 Codex CLI 的批量安装方式效率高得多。**

6.4 前置依赖

PowerSkills 是**建立在 PowerMCP 之上**的，所以必须先装好 PowerMCP：

```
pip install powermcp
powermcp install
powermcp doctor
```

某些技能的脚本有额外依赖（各技能目录下的 `requirements.txt`）。特别注意：

- **potpourri** 需要 `pandapower<3.5` 的固定版本
- **surge** 需要 Python 3.12–3.14 与系统级 HiGHS/Ipopt 库

七、实战示例

示例 1：完整的"发现问题 → 定位 → 处置"链路

这是最能体现 PowerSkills 价值的场景。

你：加载 case39.json，跑一下潮流和 N-1，看看有没有问题。

【软件技能：pandapower 被激活】

- ① load_network("case39.json") → 39 母线、34 线路、12 变压器、10 发电机
- ② get_network_info() → 确认平衡机、开关状态、额定值字段
- ③ run_power_flow() → 收敛
报告：min/max res_bus.vm_pu、最重载线路
- ④ run_contingency_analysis() → 按故障引起的违规排名

模型输出（注意所有数字都被报出来了）：

基态潮流：收敛

- L17 负载率 105.5%、L16 负载率 103.3%（限值 100%）
- 母线 7 电压 0.9473 pu、母线 6 电压 0.9479 pu（下限 0.95 pu）

N-1 分析：

- L7 开断 → L6 达 119.3%，母线 7 降至 0.9396 pu
- L12 开断 → L8 达 119.3%、L17 达 111.1%
- L20 开断 → L4 达 116.2%

技能自动升级（每一步都有数值依据）：

母线 7 电压 0.9473 pu < 0.95 → 升级至 voltage-violation-mitigation
L17 负载率 105.5% > 100% → 升级至 thermal-overload-mitigation
L7/L12/L20 均有约束性违规 → 升级至 contingency-mitigation

按优先级：电压 → 热 → 故障

后续处置建议（按 voltage-violation-mitigation 的动作顺序）：

1. 确认违规真实：案例已求解、限值正确、发电机 Q 限值合理、AVR 正常
2. 优先使用现有无功资源：
 - 调整发电机 G6/G7/G8/G9 的电压设定值（基准构建时各下调了 0.01 pu）
 - 检查是否有空闲 LTC 可用
 - 投切并联电容器组
3. 若仍不足，减少无功输送：再调度有功更靠近负荷区
4. 检查加固需求

关键观察：三个故障都指向**母线 7 附近的同一薄弱区**，这提示可以按 contingency-mitigation 的"分组"思路，用一个预防性动作覆盖整族故障。

示例 2：用 PTDF 指导再调度（带计算过程）

场景：线路 A-B 在某个 N-1 后过载至 105%。

智能体应当给出的完整推理（这是 thermal-overload-mitigation 的算例模式）：

问题：Line A-B 过载至 105%（额定值基准：正常额定值）

步骤 1 | 确认：额定值集正确、停运状态正确、基态已求解 ✓

步骤 2 | 查灵敏度：

`compute_ptdf(...)`

→ G3 到 G5 的转移在线路 A-B 上的 PTDF = -0.18 MW/MW

步骤 3 | 计算所需转移量：

目标：卸载约 7 MW 使其降至约 94%

需要转移量 = $7 / 0.18 \approx 39 \text{ MW} \approx 40 \text{ MW}$

步骤 4 | 执行：

redispatch：G3 减少 40 MW，G5 增加 40 MW

步骤 5 | 验证（关键）：

重跑【全部受监视故障集】，而非仅最严重的那个

→ 确认无新违规

结果：A-B 降至 94%，其他工况未受影响 ✓

注意步骤 5。原文的规则是："Validate a candidate fix on **all critical monitored contingencies**, not just the worst case."（在所有关键受监视故障上验证候选方案，不只是最坏情况。）

示例 3：一个故障族，一个动作

场景：N-1 停运 Line X-Y 造成热违规 + 电压违规。

不做分组的做法（低效）：

针对热违规 → 设计再调度方案

针对电压违规 → 设计无功方案

针对其他故障 → 再分别设计.....

结果：三个动作解决问题

正确的做法（先分组）：

步骤 1 | 列出所有违规:

Line A-B 过载至 108%

母线 C 电压跌至 0.91 pu

步骤 2 | 分组分析:

两个违规都位于同一条【东西向走廊】

→ 属于同一故障族

步骤 3 | 设计一个预防性动作覆盖整族:

- G1 → G4 转移 60 MW → 消除热违规
- 母线 C 投入 50 Mvar 电容器 → 电压恢复至 0.97 pu

步骤 4 | 重跑每个受监视故障 → 确认无新违规 ✓

这个模式的通用价值: 在真实的电网分析中, **按走廊/断面/薄弱区域分组**是资深工程师的本能反应。它能大幅减少所需措施数量, 也更容易被调度部门接受。

示例 4: 识别"物理不可行"而非"数值失败"

场景: 潮流在接近某个高负荷水平时反复不收敛。

初级做法 (无效循环):

尝试平启动 → 失败

尝试热启动 → 失败

调整 max_iteration → 失败

放宽 tolerance → 失败

换算法 → 失败

(反复折腾求解器设置)

正确做法 (按 convergence-failure-mitigation):

步骤 1 | 排除数据问题

孤立/失电母线？直路阻抗异常？平衡机缺失或错位？✓ 均正常

步骤 2 | 检查孤岛与平衡覆盖

`get_islands()` → 单一孤岛，平衡机覆盖正常 ✓

步骤 3 | 简化求解再逐步恢复

锁定分接头/并联/移相器 + 放宽 Q 限值 → 求解成功 ✓

→ 说明存在【控制交互】或【数值脆弱性】

步骤 4 | 逐步恢复控制，找到失效点

步骤 5 | 若仅在目标应力水平附近失败 →

判定为【可行性边界】，而非数值 bug

→ 正确措施：降低输电/负荷、增加无功支撑

→ 并标记可能的电压崩溃临近

交付时的诚实说明：

失败分类：物理不可行（接近电压崩溃边界）

具体改动：无需修复求解器设置

遗留警告：步骤 3 中锁定了控制并放宽了 Q 限值，

若要用其结果，必须恢复真实性后重新求解

这个示例的价值在于展示了"诚实报告"：技能明确要求区分失败类型，并**主动标注遗留的真实性警告**——"用锁定控制求解出来的案例不算完成"。

示例 5：DER 承载能力评估（含反直觉要点）

场景：某配电网馈线要接入光伏，评估承载力。

步骤 1 | 用正确快照确认约束类型

△ 不是峰值负荷 —— 而是【最小负荷 / 最大发电】快照

用 `OpenDSS: compile_and_solve("feeder.dss")`

检查：沿馈线电压抬升、变电站反向潮流、元件负载率、调压器行为

步骤 2 | 先查逆变器能力（最便宜的杠杆）

- Volt-VAR 曲线
- Volt-Watt 曲线
- 固定功率因数设置
- 出力限制

步骤 3 | 再调馈线控制

- 调压器与 LTC 设定值（含反向功率模式）
- 电容器投切与新潮流模式的协调

步骤 4 | 复核保护

- DER 馈入导致的继电器范围缩减
- 同情跳闸
- 熔断器-重合器配合
- 高 DER 出力下的防孤岛

步骤 5 | 最后考虑加固

- 换线 / 加装调压器 / 相平衡 / 负荷或 DER 转移
- 量化每个升级带来的承载能力增益

验证要求（容易遗漏）：

不能只验证单个快照，要跨【日曲线】验证：

修好正午的 Volt-VAR 设置，可能在黄昏时行为异常

示例 6：收敛失败的"最大失配母线"线索

场景：潮流不收敛。

错误做法：直接去调求解器全局设置

正确做法：

- ① 看失配报告，找【最大失配母线】
 - 那是问题所在的位置，从那里开始查
- ② 一次只改一个求解器设置，并记录是什么最终解决了问题
- ③ 若用锁定控制或放宽 Q 限值求解成功，
 - 报告时必须标注这个真实性警告
- ④ 若仅在接近目标应力时失败
 - 判定为可行性边界，而非数值问题

八、贡献新技能

PowerSkills 提供了 **Skill Creator** 工具（[skill-creator/](#)）用于编写与验证技能。

8.1 贡献方向

官方明确欢迎两类贡献：

1. 为现有 PowerMCP 软件面提供更好的渐进式披露（[powerskills-tool](#)）
2. 使用这些软件工具的更好处置剧本（[powerskills-engineering](#)）

8.2 编写规范

要求	说明
Keep SKILL.md concise and procedural	保持简洁、面向过程
软件技能	按"加载/打开 → 检查 → 求解 → 修改 → 高级研究"暴露工具， 结尾必须有升级触发器表
处置技能	从观测到的问题入手， 按资深工程师实际会尝试的顺序 列出纠正动作
资源引用	用技能相对路径（ scripts/... 、 references/... ），保证独立安装时仍可移植

8.3 提交前验证

```
python skill-creator/scripts/quick_validate.py powerskills-tool/skills/<skill>
python skill-creator/scripts/quick_validate.py powerskills-engineering/skills/<skill>
```

九、实践建议与注意事项

9.1 上手路径

阶段	做什么
第一步	装好 PowerMCP 作为前提 (<code>powermcp install + doctor</code>)
第二步	先装 1 个软件技能 (推荐 <code>pandapower</code> , 自带 <code>case39.json</code> 测试网络)
第三步	装互补的处置技能 (<code>voltage-violation-mitigation</code> + <code>thermal-overload-mitigation</code>)
第四步	跑通示例 1 的完整链路, 确认升级触发器正常工作
第五步	再逐步扩充到全套技能

不要一上来装全部 21 个技能。技能越多, description 匹配的干扰也越多。先装一组互补的, 验证效果后再扩。

9.2 注意事项

事项	说明
必须先装 PowerMCP	PowerSkills 是技能层, 不提供任何工具 , 它只教"怎么用工具"
商业软件仍需许可	涉及 PowerWorld/PSS/E 的技能, 底层仍依赖 PowerMCP 的相应配置
potpourri 版本约束	需要 <code>pandapower<3.5</code>
surge 环境要求	Python 3.12~3.14 + 系统级 HiGHS/Ipopt
Desktop 上传繁琐	需要逐个技能打包上传; 优先用 Claude Code / Codex CLI
技能不是硬约束	技能是"指导", 不是强制的程序化流程控制——模型理论上可以不遵守

最后一点值得诚实说明: 技能通过 description 匹配和文本指导生效, 属于**软约束**。它能让模型大概率遵循正确的流程, 但不像代码那样强制。所以**关键结论仍需人工复核**。

9.3 与 PowerMCP 的分工

	PowerMCP	PowerSkills
提供	15 款软件的工具接口	21 个技能说明书
形态	Python 服务器代码	Markdown 文档 + 脚本
回答	"能不能调用"	"该怎么用、何时升级"
能独立工作吗	能 (但没有经验指导)	不能 (无工具可用)

十、为什么这套设计值得关注

10.1 隐性知识的显式化

PowerSkills 的十本处置手册, 本质上是把**过去只存在于资深工程师头脑中的经验**, 转化为可检查、可版本控制、可评测的结构化文本。

举例来说，以下这些判断原本很难传递：

- "DER 研究要看最小负荷最大发电快照，不是峰值负荷"
- "短路校核不能只查三相故障，单相接地可能更严重"
- "用锁定控制求解出来的案例不算完成"
- "区分'数学失败了'和'运行点不存在'"
- "把'安全并网所需'和'满发所需'分开"

这些是真正的行业经验，不是能从教科书里直接查到的。把它们写下来并让 AI 遵循，是这套方案最有价值的贡献之一。

10.2 为 AI 输出赋予可验证性

通过强制"报出具体数字"和"数值驱动的升级"，PowerSkills 把 AI 的结论从模糊的自然语言变成了可核查的工程陈述。

不可验证	可验证
"存在违规"	"母线 7 电压 0.9473 pu < 0.95 pu 下限"
"存在过载"	"L17 负载率 105.5% > 100%"
"系统不稳定"	"主导模式 0.42 Hz，阻尼比 3.1% < 5% 判据"

这一点对于AI在电力这类安全关键领域的落地至关重要。

10.3 科研价值

对研究者而言，PowerSkills 提供了一份罕见的开放专家知识语料：

- 每本手册都按"资深工程师的实际尝试顺序"组织动作
- 每个软件技能都有可测量的升级触发器（可用于量化评测）
- 可以设计对照实验：**同一基准任务下，有/无 PowerSkills 的表现差异**
- surge 技能自带 evals/ 评测集，是现成的实验起点

附录：资源索引

资源	链接
仓库主页	https://github.com/Power-Agent/PowerSkills
PowerMCP（前置依赖）	https://github.com/Power-Agent/PowerMCP
Agent Skills 格式	https://agentskills.io/
Claude Code 插件文档	/plugin 命令
贡献指南	https://power-agent.github.io/

技能完整清单

软件技能（11）：`pandapower`、`pypsa`、`surge`、`powerworld`、`psse`、`pslf`、`opendss`、`andes`、`egret`、`ltspice`、`potpourri`

处置技能（10）：`voltage-violation-mitigation`、`thermal-overload-mitigation`、`contingency-mitigation`、`dynamic-stability-mitigation`、`frequency-response-mitigation`、`convergence-failure-`

mitigation、short-circuit-mitigation、interconnection-impact-mitigation、der-hosting-capacity-mitigation、operations-planning-mitigation

核心团队

Qian Zhang、Matheus Duarte、Muhy Eddin Za'ter、Drew Gray

维护方：哈佛大学 SEAS 电力与人工智能倡议（Power and AI Initiative, PAI）

本文档基于 PowerSkills 仓库全部 21 个 SKILL.md 原文与官方文档整理，生成时间：2026 年 9 月 19 日